

Evidencia 1: Un lenguaje para definir automatas

Clase:	Implementación de métodos computacionales
Profesor:	Maria Valentina Narváes Teran
Periodo y año:	FJ2026
Estudiantes:	Jorge Arturo Montiel Navarro David Cantú Daniel Camacho
Matricula:	A01278612 A0 A0
Campus:	Monterrey
Fecha de entrega:	Mayo 24, 2026

1 Introducción

La creación de un lenguaje específico para definir autómatas es una tarea que combina aspectos de teoría de la computación, diseño de lenguajes y desarrollo de software. Este proyecto tiene como objetivo desarrollar un lenguaje que permita a los usuarios describir autómatas de manera clara y concisa, facilitando su análisis y simulación. Además, se implementará un sistema de simulación que permita a los usuarios probar sus autómatas con diferentes entradas, proporcionando retroalimentación inmediata sobre su comportamiento.

2 Diseño y metodología

3 Estructura del Lenguaje

El lenguaje permite definir autómatas finitos deterministas (DFA). Un programa es siempre un único bloque `Automaton` con cinco secciones obligatorias: estados, alfabeto, estado inicial, estados finales y transiciones.

3.1 Sintaxis general

```
Automaton <identificador> [  
  states      : [ <q>, ... ]  
  alphabet    : [ <s>, ... ]  
  start       : <q>  
  end         : [ <q>, ... ]  
  transitions : [  
    <q_origen> :: <simbolo> :: <q_destino>,  
    ...  
  ]  
]
```

3.2 Ejemplo concreto

El siguiente autómata acepta únicamente la cadena `ab`:

```
Automaton dfa [  
  states      : [ q0, q1, q2 ]  
  alphabet    : [ a, b ]  
  start       : q0  
  end         : [ q2 ]  
  transitions : [  
    q0 :: a :: q1,  
    q1 :: b :: q2  
  ]  
]
```

3.3 Categorías de tokens

Tipo	Patrón (regex)	Ejemplo
rw-automata	Automaton	Automaton
rw-states	^states	states
rw-alphabet	^alphabet	alphabet
rw-start	^start	start
rw-end	^end	end
rw-transitions	^transitions	transitions
stateId	^q[0-9]+	q0, q12
alphabet-symbol	^[a-zA-Z0-9]	a, 0
identifier	^[a-zA-Z_][a-zA-Z0-9_]*	dfa
transition-sybol	^::	::
dots	^:	:
coma	^,	,
right-straight-parenthesis	^]	]
left-straight-parenthesis	^[	[
blank_space	^[]+	espacios
newline	^\n	salto de línea

Nota: cuando dos patrones tienen el mismo prefijo (p. ej. `:` y `::`), el tokenizador siempre selecciona la coincidencia más larga, por lo que `::` se clasifica como `transition-sybol` y nunca como dos `dots` consecutivos.

4 Analizador Léxico

El analizador léxico está implementado en `lexer.rkt` y exporta la función `Tokenizer`. Su responsabilidad es convertir la cadena de texto de entrada en un flujo lineal de tokens que el parser pueda consumir.

4.1 Coincidencia máxima (*maximal munch*)

En cada posición de la cadena, el tokenizador aplica *todas* las expresiones regulares de forma simultánea y se queda con la que produce la subcadena más larga. Este principio se denomina *maximal munch* y garantiza que patrones como `::` tienen prioridad sobre `:`.

El proceso se implementa en dos funciones auxiliares:

- `getMatch (label-regex, str)`: aplica la regex al inicio de `str` y devuelve la tripla (`label`, `longitud`, `lexema`). Si no hay coincidencia retorna (`"none"`, `0`, `""`).
- `getMax (matches)`: recibe la lista de resultados de `getMatch` para todas las reglas y devuelve la tripla con mayor longitud.

4.2 Gestión de líneas y errores léxicos

El tokenizador mantiene dos acumuladores a lo largo de la recursión:

- `current-line-tokens`: tokens recolectados en la línea en curso.
- `token-stream`: tokens consolidados de líneas ya completas.

Al encontrar un `newline`, los tokens de la línea actual pasan al `token-stream`. Si en cambio se produce un error léxico (ninguna regex coincide), la función `flush-line` descarta el resto de la línea y los tokens parciales acumulados en `current-line-tokens` se descartan también, impidiendo que fragmentos inválidos lleguen al parser.

4.3 Salida

`Tokenizer` devuelve una lista de dos elementos:

```
(list token-stream error-line)
```

- `token-stream`: lista de pares (tipo `lexema`), por ejemplo:
`((("rw-automata" "Automaton") ("identifier" "dfa") ("dots" ":") ...))`
- `error-line`: `#f` si el análisis fue exitoso, o la subcadena que causó el error en caso contrario.

5 Analizador Sintáctico (Descenso Recursivo)

El analizador sintáctico está implementado en `parser.rkt` y emplea la técnica de **descenso recursivo predictivo LL(1)**. Sigue estrictamente las reglas de programación funcional: no se usa `set!` ni ninguna otra forma de mutación.

5.1 Gramática formal

Cada regla de producción corresponde directamente a una función en el código. La notación usa `|` para alternativas y los tokens aparecen en *tipográfica*.

```
automaton
→ rw-automata identifier [
    statesDefinition alphabetDefinition
    startDefinition endDefinition
    transitionsDefinition ]

statesDefinition → rw-states dots [ stateId statesPrime
statesPrime      → , stateId statesPrime | ]

alphabetDefinition → rw-alphabet dots [ alphabet-symbol alphabetPrime
alphabetPrime      → , alphabet-symbol alphabetPrime | ]

startDefinition → rw-start dots stateId

endDefinition → rw-end dots [ stateId endPrime
endPrime      → , stateId endPrime | ]

transitionsDefinition → rw-transitions dots [ transition transitionsPrime
transitionsPrime      → , transition transitionsPrime | ]
transition             → stateId :: alphabet-symbol :: stateId
```

5.2 Encadenamiento funcional de resultados

Cada función de análisis recibe y devuelve la misma tripla:

```
(syntax-X tokens auto errors)
→ (list remaining-tokens auto errors)
```

- `tokens`: lista de tokens aún no consumidos.
- `auto`: hash Racket con el autómata en construcción (inmutable; cada paso produce un hash nuevo con `hash-set`).
- `errors`: lista acumulada de mensajes de error.

`syntax-automaton` encadena secuencialmente los cinco sub-analizadores pasando el resultado de uno como entrada del siguiente:

```
(define states-r (syntax-statesDefinition body-tokens auto errors))
(define alpha-r (syntax-alphabetDefinition tokens-r1 auto-r1 err-r1))
(define start-r (syntax-startDefinition tokens-r2 ...))
...
```

5.3 Estructura del autómata resultante

Al finalizar sin errores el parser produce el siguiente hash:

Clave	Tipo	Descripción
'name	string	Identificador del autómata
'states	lista de strings	Conjunto de estados Q
'alphabet	lista de strings	Alfabeto de entrada Σ
'start	string	Estado inicial q_0
'end	lista de strings	Estados de aceptación $F \subseteq Q$
'transitions	hash de hashes	Función de transición δ

La función de transición se almacena como un hash anidado donde `transitions[q][s] = q'` representa $\delta(q, s) = q'$:

```
#hash((q0 . #hash((a . q1)))
      (q1 . #hash((b . q2)))
      (q2 . #hash((0 . q0))))
```

5.4 Manejo de errores sintácticos

Cada función genera mensajes con el formato:

```
"Expected <esperado>, received <encontrado>"
```

Los errores se acumulan sin detener el análisis, lo que permite reportar múltiples problemas en una sola pasada. La función pública `Recursive-descent` devuelve:

- `(list #t auto)` — análisis exitoso.
- `(list #f errors)` — uno o más errores; `errors` es la lista de mensajes que el servidor retorna al cliente con código HTTP 400.

5.5 Validación semántica

Una vez que el análisis sintáctico concluye sin errores, `Recursive-descent` invoca a `validate-automaton` para verificar la coherencia semántica del autómata construido. Los errores sintácticos impiden que este paso se ejecute: la validación semántica solo tiene sentido sobre un autómata bien formado estructuralmente.

Las comprobaciones realizadas son:

- El estado inicial (`start`) debe estar declarado en `states`.
- Cada estado de aceptación en `end` debe estar declarado en `states`.
- En cada transición (`from, sym, to`):
 - `from` debe ser un estado declarado.
 - `sym` debe pertenecer al alfabeto declarado.
 - `to` debe ser un estado declarado.

Los mensajes de error semántico incluyen el identificador problemático entre comillas simples, por ejemplo:

```
"Transition: destination 'q99' is not a declared state"
"Transition: symbol 'x' is not in the alphabet"
```

Esta convención permite que la interfaz web localice y seleccione el término incorrecto directamente en el área de texto del editor (véase la siguiente sección).

5.6 Visualización de errores en la interfaz

Tanto el botón **Run** como el botón **Simulate** invocan **Recursive-descent** antes de proceder. Si se detecta cualquier error (sintáctico o semántico), el servidor responde con HTTP 400 y un objeto JSON de la forma:

```
{ "errors": [ "mensaje 1", "mensaje 2", ... ] }
```

El cliente reacciona de la siguiente manera:

- Oculta la sección de resultados (flujo de tokens, grafo y simulador) de modo que el usuario solo ve los errores, sin información parcial que pueda ser confusa.
- Muestra el panel **Errors** con la lista numerada de mensajes.
- Aplica un borde rojo al área de texto del editor como indicación visual inmediata de que la definición contiene problemas.
- Para el primer error que contenga un identificador entre comillas simples, busca ese término en el texto del editor, lo selecciona con `setSelectionRange` y desplaza el área de texto hasta situarlo en pantalla. Esto permite al usuario ver exactamente en qué parte de la definición se encuentra el elemento no declarado.

Cuando la siguiente ejecución de Run o Simulate tiene éxito, el panel de errores se oculta, el borde rojo desaparece y se muestran los resultados normales.

6 Pruebas

Para validar el correcto funcionamiento del sistema se definen dos autómatas que reconocen el mismo lenguaje: cadenas sobre $\{a, b\}$ que **terminan con la subcadena** `ab`. Esto permite comparar directamente la definición determinista y la no determinista.

6.1 DFA — Autómata Finito Determinista

El DFA resuelve el problema de forma explícita: necesita estados adicionales para registrar si el sufijo visto hasta ahora puede continuar hacia `ab`, y transiciones de *retroceso* cuando la secuencia se rompe.

```
Automata dfa [
  states: [q0, q1, q2]
  alphabet: [a, b]
  start: q0
  end: [q2]
  transitions: [q0::a::q1, q0::b::q0, q1::a::q1, q1::b::q2, q2::a::q1, q2::b::q0]
]
```

Lógica de los estados:

- `q0` — estado inicial; no se ha visto ningún prefijo relevante.

- **q1** — se acaba de leer una **a**; si llega **b** se acepta.
- **q2** — estado de aceptación; se ha leído **ab**. Si llega **a** hay que volver a **q1**; si llega **b**, a **q0**.

Cadena de entrada	Resultado esperado
ab	ACCEPTED
aab	ACCEPTED
bab	ACCEPTED
b	REJECTED
ba	REJECTED
aba	REJECTED

6.2 NFA — Autómata Finito No Determinista

El NFA expresa el mismo lenguaje con menos transiciones gracias al no determinismo: el estado **q0** tiene *dos* transiciones sobre **a** — una que permanece en **q0** (la **a** aún no inicia el sufijo final) y otra que avanza a **q1** (la **a** sí podría ser el inicio del sufijo **ab**). El simulador explora ambas ramas en paralelo (BFS sobre el conjunto de estados activos).

```
Automata nfa [
  states: [q0, q1, q2]
  alphabet: [a, b]
  start: q0
  end: [q2]
  transitions: [q0::a::q0, q0::a::q1, q0::b::q0, q1::b::q2]
]
```

Lógica de los estados:

- **q0** — estado inicial; consume cualquier prefijo arbitrario.
- **q1** — se acaba de “apostar” por el inicio del sufijo **ab**; solo avanza si llega **b**.
- **q2** — estado de aceptación; se ha confirmado el sufijo **ab**.

Cadena de entrada	Resultado esperado
ab	ACCEPTED
aab	ACCEPTED
bab	ACCEPTED
b	REJECTED
ba	REJECTED
aba	REJECTED

6.3 Comparación

Ambas definiciones reconocen exactamente el mismo lenguaje pero difieren en su estructura: el DFA requiere 6 transiciones y gestión explícita de retrocesos, mientras que el NFA necesita solo 4 transiciones al delegar la exploración de alternativas al simulador. Esto ilustra la equivalencia teórica entre DFA y NFA y el coste práctico de determinar un autómata.

7 Prerequisitos

7.1 Graphviz

El sistema genera una imagen del autómata a partir de un archivo `.dot`. Para ello se apoya en **Graphviz**, una herramienta de código abierto que convierte descripciones textuales de grafos en imágenes (PNG, SVG, PDF, etc.). Sin Graphviz instalado, el servidor no puede producir la visualización del DFA.

- **Windows:** descargar el instalador desde <https://graphviz.org/download/> y agregar el directorio `bin/` al PATH del sistema.

- **Linux (Debian/Ubuntu):**

```
sudo apt-get install graphviz
```

- **macOS:**

```
brew install graphviz
```

7.2 Docker

El proyecto se distribuye y ejecuta dentro de un contenedor **Docker**. Las razones principales son:

- **Portabilidad:** Racket, Graphviz y todas sus dependencias quedan encapsuladas en la imagen. Cualquier máquina con Docker instalado puede levantar el servidor con un único comando, sin necesidad de instalar Racket ni configurar el entorno manualmente.
- **Reproducibilidad:** la imagen fija las versiones exactas de todas las herramientas, eliminando el problema de *“funciona en mi máquina”*.
- **Aislamiento:** el servidor corre en su propio espacio de red y sistema de archivos, sin interferir con el resto del sistema operativo anfitrión.
- **Despliegue simplificado:** el mismo artefacto que se prueba localmente es el que se despliega en producción, sin pasos de reconfiguración.

7.3 Configuración de red y documentación del servlet

Al desplegar el servidor Racket dentro de Docker surgió un problema no obvio: `serve/servlet` enlaza el socket en 127.0.0.1 (loopback) de forma predeterminada. Dentro de un contenedor, esa dirección es *local al contenedor*, por lo que el mecanismo de port-forwarding de Docker (`-p 8080:8080`) no puede alcanzarla desde el anfitrión.

La solución, encontrada en la documentación oficial del web-server de Racket, es pasar el parámetro `#:listen-ip #f`, que indica a Racket que escuche en *todas* las interfaces de red (0.0.0.0):

```
(serve/servlet start
  #:port 8080
  #:listen-ip #f ; escucha en 0.0.0.0, no solo loopback
  ...)
```

Sin este parámetro el contenedor arranca sin errores pero las peticiones del anfitrión (o de internet) nunca llegan al proceso Racket.

La misma documentación fue necesaria para comprender el uso correcto de `hash-ref` y `list-ref`, las funciones estándar de Racket para acceder a estructuras de datos inmutables:

- `(hash-ref tabla clave)` — devuelve el valor asociado a `clave` en un hash inmutable; lanza error si la clave no existe a menos que se proporcione un valor por defecto: `(hash-ref tabla clave valor-defecto)`.
- `(list-ref lista i)` — devuelve el elemento en la posición `i` (base 0) de una lista; se usó en el parser para acceder a los cinco tokens de una transición sin descomponer manualmente la lista con `car/cdr`.

```
Hello maestra :p
```